Chapitre 6 — Représentation des données et opérations bit à bit

Prérequis : chapitres 1 à 5 · **Exercices :** `E06_Representation_donnees_bit_a_bit`

Objectifs

- comprendre la représentation d'un caractère et d'un octet ;
- utiliser les types entiers non signés ;
- effectuer des opérations bit à bit en C++ ;
- créer et utiliser des masques binaires ;
- masquer et retrouver une donnée avec XOR ;
- manipuler une adresse IPv4.

Sommaire

- Chapitre 6 — Représentation des données et opérations bit à bit
  - Objectifs
  - Sommaire
  - 1. Représentation des données
    - 1.1 Les bases numériques en C++
    - 1.2 Les types entiers non signés
    - 1.3 Application : lire une trame
  - 2. Les opérateurs bit à bit
    - 2.1 Les opérateurs ET, OU et XOR
    - 2.2 Le complément NON : `~`
    - 2.3 Les décalages `<<` et `>>`
    - 2.4 Propriété du XOR et clé répétée
  - 3. Les masques binaires
    - 3.1 Créer un masque
    - 3.2 Les quatre opérations essentielles
    - 3.3 Exemple : commander des voyants
  - 4. Application : les adresses IPv4
    - 4.1 Adresse réseau et masque de sous-réseau
    - 4.2 Manipuler une adresse IPv4 en C++

1. Représentation des données

1.1 Les bases numériques en C++

En C++, un même nombre peut être écrit dans différentes bases :

Écriture	Base	Exemple
`47`	Décimale	47
`0x2F`	Hexadécimale	47
`0b00101111`	Binaire	47

Les préfixes `0x` et `0b` indiquent respectivement une écriture hexadécimale et binaire.

L'apostrophe permet de séparer les chiffres pour faciliter la lecture :

```cpp
unsigned int valeur{0b0010'1111};
```

```cpp
#include <bitset>
#include <iostream>

int main()
{
    unsigned int a{47};             // décimal
    unsigned int b{0x2F};           // hexadécimal
    unsigned int c{0b0010'1111};    // binaire

    std::cout << "Decimal     : " << a << '\n';
    std::cout << "Hexadecimal : " << std::hex << a << '\n';
    std::cout << std::dec;          // retour à l'affichage décimal
    std::cout << "Binaire     : " << std::bitset<8>{a} << '\n';

    return 0;
}
```

Résultat :

```
Decimal     : 47
Hexadecimal : 2f
Binaire     : 00101111
```

Note :

- `47`, `0x2F` et `0b00101111` sont **la même valeur**, écrite dans trois bases.
- `std::hex` reste actif tant qu'on n'écrit pas `std::dec`.
- `std::bitset<8>{valeur}` (en-tête `<bitset>`) affiche les 8 bits de poids faible de `valeur`.

1.2 Les types entiers non signés

Pour manipuler des octets et des données binaires, on utilise les types définis dans l'en-tête `<cstdint>`.

Type	Taille	Valeurs possibles
`std::uint8_t`	8 bits	0 à 255
`std::uint16_t`	16 bits	0 à 65 535
`std::uint32_t`	32 bits	0 à 4 294 967 295

Le préfixe `uint` signifie *unsigned integer* : l'entier est non signé et ne peut pas être négatif.

Remarque : `std::uint8_t` est généralement affiché comme un caractère par `std::cout`. Pour afficher sa valeur numérique, il faut le convertir :

```cpp
std::uint8_t octet{65};

std::cout << static_cast<unsigned int>(octet) << '\n';
// Affiche 65
```

```cpp
#include <cstdint>
#include <iostream>

int main()
{
    std::uint8_t octet{65};

    std::cout << "Sans conversion : " << octet << '\n';
    std::cout << "Avec conversion : "
              << static_cast<unsigned int>(octet) << '\n';

    return 0;
}
```

Résultat :

```
Sans conversion : A
Avec conversion : 65
```

1.3 Application : lire une trame

Une trame reçue sur un réseau ou une liaison série est une suite d'octets. Ici, chaque octet est le code ASCII d'une lettre.

```cpp
#include <cstdint>
#include <iostream>
#include <vector>

int main()
{
    const std::vector<std::uint8_t> trame{
        0x42, 0x4F, 0x4E, 0x4A, 0x4F, 0x55, 0x52
    };

    for (std::uint8_t octet : trame)
    {
        std::cout << static_cast<char>(octet);
    }
    std::cout << '\n';

    return 0;
}
```

Résultat :

```
BONJOUR
```

2. Les opérateurs bit à bit

Les opérateurs bit à bit travaillent directement sur les bits des nombres, colonne par colonne.

2.1 Les opérateurs ET, OU et XOR

Opérateur	Nom	Le bit résultat vaut 1 si…
`&`	ET	les deux bits valent 1
`|`	OU	au moins un des deux bits vaut 1
`^`	OU exclusif (XOR)	les deux bits sont différents

Table de vérité

Table de vérité, pour un bit :

a	b	`a & b`	`a | b`	`a ^ b`
0	0	0	0	0
0	1	0	1	1
1	0	0	1	1
1	1	1	1	0

Sur un nombre, on applique la table à chaque colonne :

```
      ET           OU           XOR
    0101         0101         0101      (5)
  & 0011       | 0011       ^ 0011      (3)
  ------       ------       ------
    0001 (1)     0111 (7)     0110 (6)
```

```cpp
#include <bitset>
#include <iostream>

int main()
{
    unsigned int a{0b0101};   // 5
    unsigned int b{0b0011};   // 3

    std::cout << "a & b = " << std::bitset<4>{a & b} << " = " << (a & b) << '\n';
    std::cout << "a | b = " << std::bitset<4>{a | b} << " = " << (a | b) << '\n';
    std::cout << "a ^ b = " << std::bitset<4>{a ^ b} << " = " << (a ^ b) << '\n';

    return 0;
}
```

Résultat :

```
a & b = 0001 = 1
a | b = 0111 = 7
a ^ b = 0110 = 6
```

2.2 Le complément NON : `~`

L'opérateur `~` inverse **tous** les bits : les 0 deviennent des 1, les 1 deviennent des 0.

```
valeur  : 00001111   (15)
~valeur : 11110000   (240)
```

```cpp
#include <bitset>
#include <cstdint>
#include <iostream>

int main()
{
    std::uint8_t valeur{0b0000'1111};
    std::uint8_t resultat{static_cast<std::uint8_t>(~valeur)};

    std::cout << "valeur  : " << std::bitset<8>{valeur} << '\n';
    std::cout << "~valeur : " << std::bitset<8>{resultat}
              << " = " << static_cast<unsigned int>(resultat) << '\n';

    return 0;
}
```

Résultat :

```
valeur  : 00001111
~valeur : 11110000 = 240
```

Le `static_cast` permet de conserver uniquement les 8 bits du résultat.

2.3 Les décalages `<<` et `>>`

`valeur << n` décale les bits de `n` positions vers la gauche : des 0 entrent par la droite. `valeur >> n` décale les bits de `n` positions vers la droite : des 0 entrent par la gauche (pour un type non signé).

```
12      = 00001100
12 << 1 = 00011000 = 24        12 >> 1 = 00000110 = 6
12 << 2 = 00110000 = 48        12 >> 2 = 00000011 = 3
```

Pour un entier non signé :

- décaler d'une position à gauche **multiplie par 2** ;
- décaler d'une position à droite **divise par 2** (on garde la partie entière).

Les bits qui sortent du type sont **perdus**. Sur un `std::uint8_t` : `200 << 1` donne `11001000 → 10010000`, soit 144 et non 400.

```cpp
#include <bitset>
#include <cstdint>
#include <iostream>

int main()
{
    std::uint8_t valeur{12};

    std::cout << "valeur : " << std::bitset<8>{valeur}
              << " = " << static_cast<unsigned int>(valeur) << '\n';

    for (int n{1}; n <= 2; ++n)
    {
        std::uint8_t resultat{static_cast<std::uint8_t>(valeur << n)};

        std::cout << "<< " << n << "   : " << std::bitset<8>{resultat}
                  << " = " << static_cast<unsigned int>(resultat) << '\n';
    }

    for (int n{1}; n <= 2; ++n)
    {
        std::uint8_t resultat{static_cast<std::uint8_t>(valeur >> n)};

        std::cout << ">> " << n << "   : " << std::bitset<8>{resultat}
                  << " = " << static_cast<unsigned int>(resultat) << '\n';
    }

    return 0;
}
```

Résultat :

```
valeur : 00001100 = 12
<< 1   : 00011000 = 24
<< 2   : 00110000 = 48
>> 1   : 00000110 = 6
>> 2   : 00000011 = 3
```

Remarque : le nombre de positions doit être compris entre 0 et la largeur du type moins 1 (de 0 à 31 pour un entier de 32 bits). Sinon, le comportement est **indéfini**.

2.4 Propriété du XOR et clé répétée

Le XOR appliqué deux fois avec la même valeur retrouve la valeur initiale :

```
A ^ K ^ K = A
```

On peut ainsi masquer puis démasquer un octet avec une clé. Pour une clé de plusieurs octets, on répète ses octets dans l'ordre : l'octet de clé associé à l'indice `i` est `cle[i % cle.size()]`. Le XOR est ici un exercice de manipulation de données, pas un chiffrement sécurisé.

3. Les masques binaires

Un **masque binaire** est un nombre qui désigne les bits sur lesquels on veut agir. Il permet de tester, activer, désactiver ou inverser ces bits **sans toucher aux autres**.

3.1 Créer un masque

Pour créer un masque dont seul le bit numéro `n` vaut 1 :

```
1u << n
```

```
1u << 0  →  00000001
1u << 3  →  00001000
1u << 5  →  00100000
```

Pour sélectionner plusieurs bits, on combine les masques avec `|` :

```
unsigned int masque{(1u << 3) | (1u << 0)};
```

Résultat :

```
00001001
```

3.2 Les quatre opérations essentielles

Action	Instruction C++	Effet
Tester	`(etat & masque) != 0u`	vrai si au moins un bit du masque vaut 1
Activer	`etat |= masque`	met les bits du masque à 1
Désactiver	`etat &= ~masque`	met les bits du masque à 0
Inverser	`etat ^= masque`	inverse les bits du masque

Exemple sur 4 bits, avec `etat = 1001` et `masque = 0010` (bit 1) :

```
Tester     :  1001 & 0010 = 0000   → résultat nul : le bit 1 est à 0
Activer    :  1001 | 0010 = 1011   → le bit 1 passe à 1
Désactiver :  1011 & 1101 = 1001   → ~0010 = 1101 : le bit 1 passe à 0
Inverser   :  1001 ^ 0010 = 1011   → le bit 1 change d'état
```

Les autres bits ne changent jamais.

Pour vérifier que tous les bits du masque sont à 1 :

```
(etat & masque) == masque
```

3.3 Exemple : commander des voyants

Un système embarqué possède trois voyants commandés individuellement par des bits.

```
Bit      :    2        1          0
Voyant   :  Alarme   Capteur   Alimentation
```

Chaque voyant est associé à un masque.

```cpp
#include <bitset>
#include <iostream>

int main()
{
    constexpr unsigned int ALIMENTATION{1u << 0};   // 001
    constexpr unsigned int CAPTEUR{1u << 1};        // 010
    constexpr unsigned int ALARME{1u << 2};         // 100

    unsigned int etat{0u};
    std::cout << "Depart               : " << std::bitset<3>{etat} << '\n';

    etat |= ALIMENTATION;
    std::cout << "Activer alimentation : " << std::bitset<3>{etat} << '\n';

    etat |= ALARME;
    std::cout << "Activer alarme       : " << std::bitset<3>{etat} << '\n';

    if ((etat & ALARME) != 0u)
    {
        std::cout << "Test alarme          : active\n";
    }

    etat &= ~ALIMENTATION;
    std::cout << "Couper alimentation  : " << std::bitset<3>{etat} << '\n';

    etat ^= CAPTEUR;
    std::cout << "Inverser capteur     : " << std::bitset<3>{etat} << '\n';

    std::cout << "Valeur finale        : " << etat << '\n';

    return 0;
}
```

Résultat :

```
Depart               : 000
Activer alimentation : 001
Activer alarme       : 101
Test alarme          : active
Couper alimentation  : 100
Inverser capteur     : 110
Valeur finale        : 6
```

4. Application : les adresses IPv4

Une adresse IPv4 est composée de **4 octets**, soit 32 bits. Chaque nombre de la notation `192.168.1.10` est un octet (de 0 à 255).

4.1 Adresse réseau et masque de sous-réseau

Le masque indique quels bits forment la **partie réseau** (bits à 1) et quels bits forment la **partie machine** (bits à 0). Le masque `255.255.255.0` contient 24 bits à 1 : on l'écrit aussi `/24`.

Un **ET bit à bit** entre l'adresse et le masque met la partie machine à 0. On obtient l'**adresse réseau** :

```
Adresse : 11000000.10101000.00000001.00001010   192.168.1.10
Masque  : 11111111.11111111.11111111.00000000   255.255.255.0
          ---------------------------------- ET
Réseau  : 11000000.10101000.00000001.00000000   192.168.1.0
```

Un **OU** entre l'adresse et le masque inversé (`~masque`) met la partie machine à 1. On obtient l'**adresse de diffusion** (*broadcast*) :

```
Adresse   : 11000000.10101000.00000001.00001010   192.168.1.10
~Masque   : 00000000.00000000.00000000.11111111   0.0.0.255
            ---------------------------------- OU
Diffusion : 11000000.10101000.00000001.11111111   192.168.1.255
```

4.2 Manipuler une adresse IPv4 en C++

On range les 4 octets dans un seul `std::uint32_t` :

- **assembler** : on place chaque octet à sa position avec `<<`, puis on les regroupe avec `|` ;
- **extraire** : on ramène l'octet voulu à droite avec `>>`, puis on garde 8 bits avec `& 0xFF`.

```
            octet 1    octet 2    octet 3    octet 4
Bits :      31 … 24    23 … 16    15 … 8     7 … 0
Décalage :  << 24      << 16      << 8       aucun
```

```cpp
#include <cstdint>
#include <iostream>

// Affiche une adresse IPv4 en notation décimale pointée
void afficher_ipv4(std::uint32_t adresse)
{
    std::cout << ((adresse >> 24) & 0xFFu) << '.'
              << ((adresse >> 16) & 0xFFu) << '.'
              << ((adresse >> 8) & 0xFFu) << '.'
              << (adresse & 0xFFu) << '\n';
}

int main()
{
    // Assemblage de 192.168.1.10
    std::uint32_t adresse{
        (std::uint32_t{192} << 24) |
        (std::uint32_t{168} << 16) |
        (std::uint32_t{1} << 8) |
        std::uint32_t{10}
    };

    std::uint32_t masque{0xFFFFFF00};   // 255.255.255.0 (/24)

    std::uint32_t reseau{adresse & masque};
    std::uint32_t diffusion{adresse | ~masque};

    std::cout << "Adresse   : ";
    afficher_ipv4(adresse);

    std::cout << "Masque    : ";
    afficher_ipv4(masque);

    std::cout << "Reseau    : ";
    afficher_ipv4(reseau);

    std::cout << "Diffusion : ";
    afficher_ipv4(diffusion);

    return 0;
}
```

Résultat :

```
Adresse   : 192.168.1.10
Masque    : 255.255.255.0
Reseau    : 192.168.1.0
Diffusion : 192.168.1.255
```

Remarque : en hexadécimal, chaque octet s'écrit avec 2 chiffres. `0xFFFFFF00` se lit donc directement `FF.FF.FF.00`, soit `255.255.255.0`.